

C Language Tour

Student Grade Tracker — A Beginner's Annotated Program

Covers: data types · structs · enums · pointers · arrays · strings · loops · functions · dynamic memory · file I/O

Overview

This document walks through `student_tracker.c` — a single, self-contained C program that exercises the most important features of the C language. Instead of contrived toy snippets, every concept appears in a real working context: a command-line student grade manager.

The program lets you:

- Add students with a name and score
- View the full roster
- Search for a student by name
- Calculate and display class statistics
- Save the roster to a file and load it back

Compile & Run

```
gcc -Wall -Wextra -std=c11 -o tracker student_tracker.c
./tracker
```

► Flags explained

`-Wall -Wextra`: enable all warnings (always use these). `-std=c11`: compile as C11. `-o tracker`: name the output file.

Concept 1 — `#include` & `#define` (Preprocessor)

The preprocessor runs before the compiler. Every line starting with `#` is a directive — not C code, but instructions to the preprocessor to perform text substitution.

```
#include <stdio.h>    // pull in standard I/O declarations
#include <stdlib.h>    // malloc, free, exit
#include <string.h>    // strlen, strcpy, strcmp
#include <ctype.h>     // toupper, isdigit, ...

#define MAX_NAME      64    // max chars in a name (no ;)
#define SAVE_FILE      "students.dat"
#define PASSING_SCORE 60.0  // replaced everywhere at compile time
```

Directive	What it does
<code>#include <header></code>	Inserts the contents of a system header file
<code>#include "file.h"</code>	Inserts a local header you wrote
<code>#define NAME value</code>	Replaces every occurrence of NAME with value before compiling

 Warning

`#define` is pure text substitution — there is no type checking. For typed constants in modern C, prefer `const int MAX = 64;` instead.

Concept 2 — enum (Named Integer Constants)

An enum assigns human-readable names to integer values. Without enums you'd write `switch(choice) { case 1: ... case 2: ... }` — impossible to read. With enums the intent is self-documenting.

```
typedef enum {  
    ADD    = 1,    // explicitly start at 1 so menu numbers match  
    VIEW   = 2,    // subsequent values auto-increment  
    SEARCH = 3,  
    SAVE   = 4,  
    LOAD   = 5,  
    STATS  = 6,  
    QUIT   = 7  
} MenuChoice;  
  
// Usage  
MenuChoice m = ADD;  
if (m == QUIT) { ... }
```

The typedef wraps the enum so we can write MenuChoice instead of enum MenuChoice everywhere.

Key Idea

Enums don't create a new type at the machine level — they're just ints. But they make your code self-documenting and prevent magic numbers.

Concept 3 — struct (Custom Data Types)

A struct groups related variables into a single named type. Think of it as a lightweight class with only data — no methods.

```

typedef struct {
    char    name[MAX_NAME]; // fixed-size char array – a C string
    int     id;              // student ID
    double  score;           // exam score 0.0–100.0
    char    grade;          // letter grade: A B C D F
} Student;

// Create a Student on the stack (local variable)
Student alice;
strcpy(alice.name, "Alice"); // use . to access fields
alice.score = 88.5;

// Access fields through a pointer with ->
Student *p = &alice;
printf("%s scored %.1f\n", p->name, p->score);
// p->name is shorthand for (*p).name

```

Syntax	Meaning
<code>s.field</code>	Access field directly on a struct value
<code>p->field</code>	Access field through a pointer (shorthand for <code>(*p).field</code>)
<code>sizeof(Student)</code>	How many bytes one Student occupies in memory

Concept 4 — Functions & Prototypes

C requires every function to be declared before it is called. A prototype is the declaration (signature only) — it tells the compiler the return type and parameter types. The definition (body) can appear later.

```
// — PROTOTYPES at top of file —————
char calculateGrade(double score);
void printStudent(const Student *s); // const = won't modify *s
void addStudent(Student **roster, int *count, int *capacity);

// — DEFINITION (full body) anywhere below —————
char calculateGrade(double score) {
    if (score >= 90.0) return 'A';
    else if (score >= 80.0) return 'B';
    else if (score >= 70.0) return 'C';
    else if (score >= 60.0) return 'D';
    else return 'F';
}
```

Modifiers on parameters:

- `const Student *s` — pointer to data we promise not to modify (read-only access)
- `Student **roster` — pointer to a pointer, needed when the function must change the pointer itself (e.g. after `realloc`)

Concept 5 — Pointers (The Heart of C)

A pointer is a variable that holds a memory address. Pointers let functions modify variables in their caller, build dynamic data structures, and work efficiently with large data.

```

int x = 42;

int *p = &x;    // & = address-of: p now holds x's address

printf("%d\n", x);    // 42 - value
printf("%p\n", p);    // 0x7fff... - the address itself
printf("%d\n", *p);    // 42 - * dereferences: value at that address

*p = 100;          // write through the pointer - x is now 100

// Pass-by-pointer: let a function modify count in the caller
void increment(int *n) {
    (*n)++;    // parentheses: dereference n, then increment
}

increment(&x);    // x is now 101

```

Operator	Name	What it does
<code>&var</code>	Address-of	Gives the memory address of var
<code>*ptr</code>	Dereference	Reads or writes the value at the address ptr holds
<code>ptr->field</code>	Arrow	Dereference ptr and access field (same as <code>(*ptr).field</code>)
<code>ptr++</code>	Pointer arithmetic	Advance ptr by <code>sizeof(*ptr)</code> bytes

⚠ Common mistake

Never dereference a NULL or uninitialized pointer. Always initialize pointers: `int *p = NULL;` and check if `(p != NULL)` before dereferencing.

Concept 6 — Arrays & Strings

Arrays are contiguous blocks of same-type elements. In C, `array[i]` is exactly `*(array + i)` — the name decays to a pointer to the first element.

```
int scores[5] = {90, 85, 78, 92, 88}; // fixed-size, stack allocated
scores[0] = 100; // zero-indexed

// Iterate — track the size yourself; C arrays have no .length
int n = sizeof(scores) / sizeof(scores[0]); // = 5
for (int i = 0; i < n; i++)
    printf("%d ", scores[i]);
```

Strings

C has no String type. A string is a char array with a `'\0'` (null terminator) at the end. All string functions from `<string.h>` rely on this terminator.

```
char name[64] = "Alice"; // stored as: A l i c e \0 ??? ???

strlen(name)           // 5   (chars before \0, not counting it)
strcpy(dest, src)       // copy — dest must be large enough!
strncpy(dest, src, n)   // safer: copy at most n chars
strcmp(a, b)            // 0=equal, <0=a<b, >0=a>b
strcat(dest, src)       // append src onto dest
strcspn(s, "\n")        // chars until any char in second arg

// Strip fgets newline (used throughout the program)
s[strcspn(s, "\n")] = '\0';
```

⚠ Critical rule

NEVER use `==` to compare strings. It compares pointer addresses, not content. Always use `strcmp(a, b) == 0` to test equality.

Concept 7 — Control Flow

if / else if / else

```
if (score >= 90.0)           return 'A';
else if (score >= 80.0)      return 'B';
else if (score >= 70.0)      return 'C';
else if (score >= 60.0)      return 'D';
else                        return 'F';
```

switch

switch jumps directly to the matching case. Without break, execution falls through to the next case — usually a bug, occasionally intentional.

```
switch (choice) {
    case ADD:    addStudent(...); break; // break exits the switch
    case VIEW:   viewAll(...);   break;
    case QUIT:
        running = 0;
        break;
    default:
        // runs if no case matches
        printf("Unknown option.\n");
}
```

Three Loop Types


```
// for – best when you know the count in advance
for (int i = 0; i < count; i++) {
    printStudent(&roster[i]);
}

// while – best when the stop condition is the focus
while (running) {
    printMenu();
    // ...
}

// do-while – always runs at least once (great for menus)
do {
    printMenu();
    scanf("%d", &choice);
} while (choice != QUIT);

// Loop control
break;    // exit the loop immediately
continue; // skip to the next iteration
```

Concept 8 — Dynamic Memory (malloc / realloc / free)

Static arrays have a fixed size determined at compile time. Dynamic memory lets you allocate exactly as much as you need at runtime, from the heap.

```
#include <stdlib.h>

// malloc: allocate n bytes - contents are uninitialized (garbage!)
int capacity = 4;
Student *roster = malloc(capacity * sizeof(Student));

if (roster == NULL) {           // ALWAYS check - malloc can fail
    fprintf(stderr, "Out of memory\n");
    exit(1);
}

// realloc: grow (or shrink) an existing block
// CRITICAL: use a temp pointer - if realloc fails it returns NULL
// but the original block is still valid and must be freed.
Student *tmp = realloc(roster, newCapacity * sizeof(Student));
if (tmp == NULL) { /* handle error, roster still valid */ }
else { roster = tmp; capacity = newCapacity; }

// free: release the memory back to the OS
free(roster);
roster = NULL; // set to NULL - prevents accidental use-after-free
```

Function	Description	Returns
<code>malloc(size)</code>	Allocate size bytes (uninitialized)	<code>void*</code> or <code>NULL</code>
<code>calloc(n, size)</code>	Allocate n*size bytes, zeroed out	<code>void*</code> or <code>NULL</code>
<code>realloc(ptr, size)</code>	Resize block at ptr to size bytes	<code>void*</code> or <code>NULL</code>
<code>free(ptr)</code>	Release the block (must be called exactly once)	<code>void</code>

⚠ Memory rules

Every malloc/calloc needs exactly one free. No free = memory leak. Two frees = crash. Accessing memory after free = undefined behavior. Use Valgrind or `-fsanitize=address` to find these bugs.

Concept 9 — File I/O

File I/O in C uses FILE* handles. fopen returns a pointer to a FILE structure managed by the OS. Always close files with fclose when done.

```
#include <stdio.h>

// — WRITING —————

FILE *fp = fopen("students.dat", "w"); // "w" = create/overwrite
if (fp == NULL) { perror("fopen"); return; }

fprintf(fp, "%d\n", count);           // like printf, but to a file
for (int i = 0; i < count; i++)
    fprintf(fp, "%s\n%d\n%.2f\n%c\n",
            roster[i].name, roster[i].id,
            roster[i].score, roster[i].grade);
fclose(fp); // flush buffer and close

// — READING —————

FILE *fp = fopen("students.dat", "r"); // "r" = read existing
if (fp == NULL) { perror("fopen"); return; }

int n;
fscanf(fp, "%d\n", &n);           // read an int from file
fgets(buf, MAX_NAME, fp);         // read a line (includes \n)
buf[strcspn(buf, "\n")] = '\0';   // strip the \n
fclose(fp);
```

Mode	Opens for	Creates if missing?	Truncates existing?
"r"	Read only	No — fails	No
"w"	Write only	Yes	Yes — erases content!
"a"	Append write	Yes	No — appends to end
"r+"	Read + write	No — fails	No
"rb"	Binary read	No — fails	No
"wb"	Binary write	Yes	Yes

Concept 10 — printf / scanf Format Specifiers

printf writes formatted output to **stdout**. **scanf** reads formatted input from **stdin**. The format string uses specifiers that start with **%**.

```
// printf examples
printf("%d", 42);           // integer:      42
printf("%05d", 42);        // zero-padded: 00042
printf("%-10s", "Alice");  // left-aligned: 'Alice   '
printf("%6.2f", 88.5);     // float w/ width: ' 88.50'
printf("%c", 'A');         // char:        A
printf("%s", "hello");     // string:      hello
printf("%p", (void*)ptr);  // pointer address: 0x7ffe...

// scanf examples - & gives scanf the address to write into
int n;      scanf("%d", &n);    // read int
double d;   scanf("%lf", &d);   // read double (%lf not %f!)
char buf[64]; scanf("%63s", buf); // read word (no & needed for arrays)
```

Specifier	Type	Example output
%d / %i	int (signed decimal)	42

Specifier	Type	Example output
%u	unsigned int	42
%ld	long	-9000000
%f	float or double (printf)	3.140000
%lf	double (scanf only)	(reads 3.14 into double)
%.2f	double, 2 decimal places	3.14
%e	scientific notation	3.140000e+00
%c	char	A
%s	char* string	Hello
%p	void* pointer	0x7ffe2a3b
%zu	size_t	8
%%	literal %	%

Concept 11 — Operators

The program uses all of C's major operator categories:

```

// Arithmetic
sum += roster[i].score;      // += accumulate
int n = *capacity * 2;      // * multiply
double avg = sum / count;    // / floating-point divide (both doubles)
int pct = pass * 100 / total; // integer division (truncates)

// Comparison
if (score >= PASSING_SCORE) // >= greater-than-or-equal
if (strcmp(a, b) == 0)      // == equal (NOT = which is assignment!)
if (p == NULL)              // test for null pointer

// Logical
if (score >= 80.0 && score < 90) // && AND - short-circuits
if (choice == QUIT || choice == 0) // || OR - short-circuits
if (!found)                      // ! NOT

// Ternary (condition ? if_true : if_false)
const char *result = (score >= 60.0) ? "PASS" : "FAIL";
(score >= 60.0) ? pass++ : fail++; // increment one counter

// Cast (convert type explicitly)
double pct = (double)pass / count * 100.0; // avoid integer division

// Bitwise (not used in tracker, but important in C)
flags |= FLAG_READ; // set a bit
flags &= ~FLAG_WRITE; // clear a bit
flags ^= FLAG_EXEC; // toggle a bit

```

Program Structure at a Glance

```

main()
|
├─ malloc()           allocate initial roster on the heap
|
└─ do { ... } while(running)  ← menu loop
    |
    ├─ printMenu()      show numbered options
    ├─ scanf()          read user's choice
    |
    └─ switch(choice)
        │
        ├─ ADD   → addStudent()    add + realloc if full
        ├─ VIEW  → viewAll()       for loop + printStudent()
        ├─ SEARCH → searchByName() strcmp scan
        ├─ SAVE  → saveToFile()    fopen("w") + fprintf
        ├─ LOAD  → loadFromFile()  fopen("r") + fgets/fscanf
        ├─ STATS → showStats()     min/max/avg/pass-rate
        └─ QUIT  → set running=0, exit loop
    |
    └─ free(roster)    release heap memory before exit

```

Exercises — Take It Further

Once the program compiles and runs, try these to solidify your understanding:

- Sort the roster by score using `qsort()` from `<stdlib.h>`
- Add a 'Delete student by ID' menu option — shift the array elements down
- Add a second struct field for a second exam score and calculate an average
- Change the fixed-size `name[MAX_NAME]` to a dynamically allocated `char*` — remember to `malloc` and `free` it
- Add an 'Edit' option that searches for a student and lets you update their score
- Write the save file in binary mode ("`wb`" / "`rb`") using `fwrite` / `fread` instead of `fprintf` / `fscanf`
- Refactor the code into two files: `students.h` (declarations) and `students.c` (implementations) — then link them

The best way to learn C is to compile, break, and fix things.